

PyroGuard AI - API v1.0.0

Scroll down for code samples, example requests and responses. Select a language for code samples from the tabs above or the mobile navigation menu.

Microservicio RESTful de Inteligencia Artificial para la predicción de incendios forestales usando Machine Learning Híbrido (K-Means + Isolation Forest) y LangChain.

Base URLs:

- </ml>

Default

read_root__get

Code samples

```
# You can also use wget
curl -X GET /ml/ \
  -H 'Accept: application/json'
```

```
GET /ml/ HTTP/1.1
```

```
Accept: application/json
```

```
const headers = {
  'Accept': 'application/json'
};

fetch('/ml/',
{
  method: 'GET',

  headers: headers
})
.then(function(res) {
  return res.json();
}).then(function(body) {
  console.log(body);
});
```

```
require 'rest-client'
require 'json'

headers = {
  'Accept' => 'application/json'
}
```

```
result = RestClient.get '/ml/',
  params: {
  }, headers: headers

p JSON.parse(result)
```

```
import requests
headers = {
  'Accept': 'application/json'
}

r = requests.get('/ml/', headers = headers)

print(r.json())
```

```
<?php

require 'vendor/autoload.php';

$headers = array(
  'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
    $response = $client->request('GET', '/ml/', array(
        'headers' => $headers,
        'json' => $request_body,
    ))
    ;
    print_r($response->getBody()->getContents());
}
catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...
```

```
URL obj = new URL("/ml/");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("GET");
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
```

```
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();
System.out.println(response.toString());
```

```
package main

import (
    "bytes"
    "net/http"
)

func main() {

    headers := map[string][]string{
        "Accept": []string{"application/json"},
    }

    data := bytes.NewBuffer([]byte{jsonReq})
    req, err := http.NewRequest("GET", "/ml/", data)
    req.Header = headers

    client := &http.Client{}
    resp, err := client.Do(req)
    // ...

}
```

GET /

Read Root

Endpoint de salud del servidor.

Example responses

200 Response

null

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	Inline

Response Schema

This operation does not require authentication

Predicciones ML

create_prediction_api_v1_predict_post

Code samples

```
# You can also use wget
curl -X POST /ml/api/v1/predict/ \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json'
```

```
POST /ml/api/v1/predict/ HTTP/1.1
```

```
Content-Type: application/json
Accept: application/json
```

```
const inputBody = '{
  "temperatura": 35.5,
  "humedad": 30,
  "viento": 15,
  "precipitacion": 0,
  "id_zona": "f93aee30-e9ed-4b8b-a1ad-e843e48e671e"
}';
const headers = {
  'Content-Type': 'application/json',
  'Accept': 'application/json'
};

fetch('/ml/api/v1/predict/',
{
  method: 'POST',
  body: inputBody,
  headers: headers
})
.then(function(res) {
  return res.json();
}).then(function(body) {
  console.log(body);
});
```

```
require 'rest-client'
require 'json'

headers = {
  'Content-Type' => 'application/json',
  'Accept' => 'application/json'
}

result = RestClient.post '/ml/api/v1/predict/',
  params: {
  }, headers: headers
```

```
p JSON.parse(result)
```

```
import requests
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json'
}

r = requests.post('/ml/api/v1/predict/', headers = headers)

print(r.json())
```

```
<?php

require 'vendor/autoload.php';

$headers = array(
    'Content-Type' => 'application/json',
    'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
    $response = $client->request('POST', '/ml/api/v1/predict/', array(
        'headers' => $headers,
        'json' => $request_body,
    ));
    print_r($response->getBody()->getContents());
}
catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...
```

```
URL obj = new URL("/ml/api/v1/predict/");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("POST");
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
```

```
        response.append(inputLine);
    }
    in.close();
    System.out.println(response.toString());
}
```

```
package main

import (
    "bytes"
    "net/http"
)

func main() {

    headers := map[string][]string{
        "Content-Type": []string{"application/json"},
        "Accept": []string{"application/json"},
    }

    data := bytes.NewBuffer([]byte{jsonReq})
    req, err := http.NewRequest("POST", "/ml/api/v1/predict/", data)
    req.Header = headers

    client := &http.Client{}
    resp, err := client.Do(req)
    // ...
}
```

POST /api/v1/predict/

Create Prediction

Body parameter

```
{
  "temperatura": 35.5,
  "humedad": 30,
  "viento": 15,
  "precipitacion": 0,
  "id_zona": "f93aee30-e9ed-4b8b-a1ad-e843e48e671e"
}
```

Parameters

Name	In	Type	Required	Description
body	body	PredictionRequest	true	none

Example responses

200 Response

```
{
  "id_prediccion": "c641af1d-4c73-4a4a-863c-f8082ef3d52a",
  "id_zona": "f93aee30-e9ed-4b8b-a1ad-e843e48e671e",
  "nivel_riesgo": "string",
  "detalles_modelos": {},
  "directiva_accion": "string",
  "fecha_evaluacion": "2019-08-24T14:15:22Z"
}
```

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	PredictionResponse
422	Unprocessable Entity	Validation Error	HTTPValidationError

This operation does not require authentication

get_prediction_history_api_v1_predict_history_get

Code samples

```
# You can also use wget
curl -X GET /ml/api/v1/predict/history \
-H 'Accept: application/json'
```

```
GET /ml/api/v1/predict/history HTTP/1.1
```

```
Accept: application/json
```

```
const headers = {
  'Accept': 'application/json'
};

fetch('/ml/api/v1/predict/history',
{
  method: 'GET',

  headers: headers
})
.then(function(res) {
  return res.json();
}).then(function(body) {
  console.log(body);
});
```

```
require 'rest-client'
require 'json'
```

```

headers = {
    'Accept' => 'application/json'
}

result = RestClient.get '/ml/api/v1/predict/history',
    params: {
    }, headers: headers

p JSON.parse(result)

```

```

import requests
headers = {
    'Accept': 'application/json'
}

r = requests.get('/ml/api/v1/predict/history', headers = headers)

print(r.json())

```

```

<?php

require 'vendor/autoload.php';

$headers = array(
    'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
    $response = $client->request('GET', '/ml/api/v1/predict/history', array(
        'headers' => $headers,
        'json' => $request_body,
    ))
};
print_r($response->getBody()->getContents());
}
catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...

```

```

URL obj = new URL("/ml/api/v1/predict/history");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("GET");

```



```
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();
System.out.println(response.toString());
```

```
package main

import (
    "bytes"
    "net/http"
)

func main() {

    headers := map[string][]string{
        "Accept": []string{"application/json"},
    }

    data := bytes.NewBuffer([]byte{jsonReq})
    req, err := http.NewRequest("GET", "/ml/api/v1/predict/history", data)
    req.Header = headers

    client := &http.Client{}
    resp, err := client.Do(req)
    // ...
}
```

GET /api/v1/predict/history

Get Prediction History

Parameters

Name	In	Type	Required	Description
skip	query	integer	false	none
limit	query	integer	false	none

Example responses

200 Response

```
[
  {
    "id_prediccion": "c641af1d-4c73-4a4a-863c-f8082ef3d52a",
    "id_zona": "f93aee30-e9ed-4b8b-a1ad-e843e48e671e",
    "nivel_riesgo": "string",
```

```
"detalles_modelos": {},  
"directiva_accion": "string",  
"fecha_evaluacion": "2019-08-24T14:15:22Z"  
}  
]
```

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	Inline
422	Unprocessable Entity	Validation Error	HTTPValidationError

Response Schema

Status Code **200**

Response Get Prediction History Api V1 Predict History Get

Name	Type	Required	Restrictions	Description
Response Get Prediction History Api V1 Predict History Get	[PredictionResponse]	false	none	none
» PredictionResponse	PredictionResponse	false	none	none
»» id_prediccion	string(uuid)	true	none	none
»» id_zona	any	false	none	none

anyOf

Name	Type	Required	Restrictions	Description
»»» <i>anonymous</i>	string(uuid)	false	none	none

or

Name	Type	Required	Restrictions	Description
»»» <i>anonymous</i>	null	false	none	none

continued

Name	Type	Required	Restrictions	Description
»» nivel_riesgo	string	true	none	Bajo, Medio, Alto, o Crítico
»» detalles_modelos	object	true	none	Resultados crudos de KMeans e Isolation Forest
»» directiva_accion	string	true	none	Texto generado por LangChain con la recomendación de mitigación
»» fecha_evaluacion	string(date-time)	true	none	none

This operation does not require authentication

Analítica Técnica (Modelos e Históricos)

obtener_incidentes_historicos_api_v1_analitica_incidentes_historicos_get

Code samples

```
# You can also use wget
curl -X GET /ml/api/v1/analitica/incidentes-historicos \
  -H 'Accept: application/json'
```

```
GET /ml/api/v1/analitica/incidentes-historicos HTTP/1.1
```

```
Accept: application/json
```

```
const headers = {
  'Accept': 'application/json'
};

fetch('/ml/api/v1/analitica/incidentes-historicos',
{
  method: 'GET',

  headers: headers
})
.then(function(res) {
  return res.json();
}).then(function(body) {
  console.log(body);
});
```

```
require 'rest-client'
require 'json'

headers = {
  'Accept' => 'application/json'
}

result = RestClient.get '/ml/api/v1/analitica/incidentes-historicos',
  params: {
  }, headers: headers

p JSON.parse(result)
```

```
import requests
headers = {
```

```

    'Accept': 'application/json'
}

r = requests.get('/ml/api/v1/analitica/incidentes-historicos', headers = headers)

print(r.json())

```

```

<?php

require 'vendor/autoload.php';

$headers = array(
    'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
    $response = $client->request('GET', '/ml/api/v1/analitica/incidentes-historicos',
    array(
        'headers' => $headers,
        'json' => $request_body,
    )
    );
    print_r($response->getBody()->getContents());
}
catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...

```

```

URL obj = new URL("/ml/api/v1/analitica/incidentes-historicos");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("GET");
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();
System.out.println(response.toString());

```

```

package main

import (
    "bytes"
    "net/http"
)

func main() {

    headers := map[string][]string{
        "Accept": []string{"application/json"},
    }

    data := bytes.NewBuffer([]byte{jsonReq})
    req, err := http.NewRequest("GET", "/ml/api/v1/analitica/incidentes-historicos",
data)
    req.Header = headers

    client := &http.Client{}
    resp, err := client.Do(req)
    // ...
}

```

```
GET /api/v1/analitica/incidentes-historicos
```

Obtener Incidentes Historicos

Devuelve los incidentes históricos registrados para cruzar en el mapa. Nota: Transforma la geometría PostGIS a texto para que el frontend la lea.

Parameters

Name	In	Type	Required	Description
limit	query	integer	false	none

Example responses

200 Response

```
null
```

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	Inline
422	Unprocessable Entity	Validation Error	HTTPValidationError

Response Schema

This operation does not require authentication

ver_umbrales_api_v1_analitica_configuracion_umbrales_get

Code samples

```
# You can also use wget
curl -X GET /ml/api/v1/analitica/configuracion-umbrales \
-H 'Accept: application/json'
```

```
GET /ml/api/v1/analitica/configuracion-umbrales HTTP/1.1
```

```
Accept: application/json
```

```
const headers = {
  'Accept': 'application/json'
};

fetch('/ml/api/v1/analitica/configuracion-umbrales',
{
  method: 'GET',

  headers: headers
})
.then(function(res) {
  return res.json();
}).then(function(body) {
  console.log(body);
});
```

```
require 'rest-client'
require 'json'

headers = {
  'Accept' => 'application/json'
}

result = RestClient.get '/ml/api/v1/analitica/configuracion-umbrales',
  params: {
  }, headers: headers

p JSON.parse(result)
```

```
import requests
headers = {
  'Accept': 'application/json'
}

r = requests.get('/ml/api/v1/analitica/configuracion-umbrales', headers = headers)
```

```
print(r.json())
```

```
<?php

require 'vendor/autoload.php';

$headers = array(
    'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
    $response = $client->request('GET', '/ml/api/v1/analitica/configuracion-umbrales',
    array(
        'headers' => $headers,
        'json' => $request_body,
    )
    );
    print_r($response->getBody()->getContents());
}
catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...
```

```
URL obj = new URL("/ml/api/v1/analitica/configuracion-umbrales");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("GET");
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();
System.out.println(response.toString());
```

```
package main

import (
    "bytes"
    "net/http"
```

```

)

func main() {

    headers := map[string][]string{
        "Accept": []string{"application/json"},
    }

    data := bytes.NewBuffer([]byte{jsonReq})
    req, err := http.NewRequest("GET", "/ml/api/v1/analitica/configuracion-umbrales",
data)
    req.Header = headers

    client := &http.Client{}
    resp, err := client.Do(req)
    // ...
}

```

GET /api/v1/analitica/configuracion-umbrales

Ver Umbrales

Example responses

200 Response

```

{
  "critico": {
    "temp": 0,
    "hum": 0
  },
  "medio": {
    "temp": 0,
    "hum": 0
  }
}

```

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	UmbralesConfig

This operation does not require authentication

actualizar_umbrales_api_v1_analitica_configuracion_umbrales_put

Code samples

```

# You can also use wget
curl -X PUT /ml/api/v1/analitica/configuracion-umbrales \
-H 'Content-Type: application/json' \
-H 'Accept: application/json'

```


PUT /ml/api/v1/analitica/configuracion-umbrales HTTP/1.1

Content-Type: application/json

Accept: application/json

```
const inputBody = '{
  "critico": {
    "temp": 0,
    "hum": 0
  },
  "medio": {
    "temp": 0,
    "hum": 0
  }
}';
const headers = {
  'Content-Type': 'application/json',
  'Accept': 'application/json'
};

fetch('/ml/api/v1/analitica/configuracion-umbrales',
{
  method: 'PUT',
  body: inputBody,
  headers: headers
})
.then(function(res) {
  return res.json();
}).then(function(body) {
  console.log(body);
});
```

```
require 'rest-client'
require 'json'

headers = {
  'Content-Type' => 'application/json',
  'Accept' => 'application/json'
}

result = RestClient.put '/ml/api/v1/analitica/configuracion-umbrales',
  params: {
  }, headers: headers

p JSON.parse(result)
```

```
import requests
headers = {
  'Content-Type': 'application/json',
  'Accept': 'application/json'
```

```

}

r = requests.put('/ml/api/v1/analitica/configuracion-umbrales', headers = headers)

print(r.json())

```

```

<?php

require 'vendor/autoload.php';

$headers = array(
    'Content-Type' => 'application/json',
    'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
    $response = $client->request('PUT', '/ml/api/v1/analitica/configuracion-umbrales',
    array(
        'headers' => $headers,
        'json' => $request_body,
    )
    );
    print_r($response->getBody()->getContents());
}
catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...

```

```

URL obj = new URL("/ml/api/v1/analitica/configuracion-umbrales");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("PUT");
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();
System.out.println(response.toString());

```

```
package main

import (
    "bytes"
    "net/http"
)

func main() {

    headers := map[string][]string{
        "Content-Type": []string{"application/json"},
        "Accept": []string{"application/json"},
    }

    data := bytes.NewBuffer([]byte{jsonReq})
    req, err := http.NewRequest("PUT", "/ml/api/v1/analitica/configuracion-umbrales",
data)
    req.Header = headers

    client := &http.Client{}
    resp, err := client.Do(req)
    // ...
}
```

PUT /api/v1/analitica/configuracion-umbrales

Actualizar Umbrales

Permite a los Analistas ajustar en caliente los pesos de temperatura y humedad.

Body parameter

```
{
  "critico": {
    "temp": 0,
    "hum": 0
  },
  "medio": {
    "temp": 0,
    "hum": 0
  }
}
```

Parameters

Name	In	Type	Required	Description
body	body	UmbralesConfig	true	none

Example responses

200 Response

null

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	Inline
422	Unprocessable Entity	Validation Error	HTTPValidationError

Response Schema

This operation does not require authentication

estacionalidad_riesgo_api_v1_analitica_estacionalidad_get

Code samples

```
# You can also use wget
curl -X GET /ml/api/v1/analitica/estacionalidad \
  -H 'Accept: application/json'
```

```
GET /ml/api/v1/analitica/estacionalidad HTTP/1.1
```

```
Accept: application/json
```

```
const headers = {
  'Accept': 'application/json'
};

fetch('/ml/api/v1/analitica/estacionalidad',
{
  method: 'GET',

  headers: headers
})
.then(function(res) {
  return res.json();
}).then(function(body) {
  console.log(body);
});
```

```
require 'rest-client'
require 'json'

headers = {
  'Accept' => 'application/json'
}
```

```
result = RestClient.get '/ml/api/v1/analitica/estacionalidad',
  params: {
  }, headers: headers

p JSON.parse(result)
```

```
import requests
headers = {
    'Accept': 'application/json'
}

r = requests.get('/ml/api/v1/analitica/estacionalidad', headers = headers)

print(r.json())
```

```
<?php

require 'vendor/autoload.php';

$headers = array(
    'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
    $response = $client->request('GET', '/ml/api/v1/analitica/estacionalidad', array(
        'headers' => $headers,
        'json' => $request_body,
    ));
    print_r($response->getBody()->getContents());
}
catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...
```

```
URL obj = new URL("/ml/api/v1/analitica/estacionalidad");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("GET");
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
```

```
while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();
System.out.println(response.toString());
```

```
package main

import (
    "bytes"
    "net/http"
)

func main() {

    headers := map[string][]string{
        "Accept": []string{"application/json"},
    }

    data := bytes.NewBuffer([]byte{jsonReq})
    req, err := http.NewRequest("GET", "/ml/api/v1/analitica/estacionalidad", data)
    req.Header = headers

    client := &http.Client{}
    resp, err := client.Do(req)
    // ...
}
```

GET /api/v1/analitica/estacionalidad

Estacionalidad Riesgo

Genera un mapa de calor mensual de riesgo histórico (agrupado por Zona y Mes) basado en los incidentes reales (incendios) de la última década.

Example responses

200 Response

null

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	Inline

Response Schema

This operation does not require authentication

Zonas Protegidas (PostGIS)

listar_zonas_simple_api_v1_zonas_simple_get

Code samples

```
# You can also use wget
curl -X GET /ml/api/v1/zonas/simple \
-H 'Accept: application/json'
```

```
GET /ml/api/v1/zonas/simple HTTP/1.1
```

```
Accept: application/json
```

```
const headers = {
  'Accept': 'application/json'
};

fetch('/ml/api/v1/zonas/simple',
{
  method: 'GET',

  headers: headers
})
.then(function(res) {
  return res.json();
}).then(function(body) {
  console.log(body);
});
```

```
require 'rest-client'
require 'json'

headers = {
  'Accept' => 'application/json'
}

result = RestClient.get '/ml/api/v1/zonas/simple',
  params: {
  }, headers: headers

p JSON.parse(result)
```

```
import requests
headers = {
  'Accept': 'application/json'
}

r = requests.get('/ml/api/v1/zonas/simple', headers = headers)
```

```
print(r.json())
```

```
<?php

require 'vendor/autoload.php';

$headers = array(
    'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
    $response = $client->request('GET', '/ml/api/v1/zonas/simple', array(
        'headers' => $headers,
        'json' => $request_body,
    ));
    print_r($response->getBody()->getContents());
}
catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...
```

```
URL obj = new URL("/ml/api/v1/zonas/simple");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("GET");
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();
System.out.println(response.toString());
```

```
package main

import (
    "bytes"
    "net/http"
)
```



```
func main() {

    headers := map[string][]string{
        "Accept": []string{"application/json"},
    }

    data := bytes.NewBuffer([]byte{jsonReq})
    req, err := http.NewRequest("GET", "/ml/api/v1/zonas/simple", data)
    req.Header = headers

    client := &http.Client{}
    resp, err := client.Do(req)
    // ...

}
```

GET /api/v1/zonas/simple

Listar Zonas Simple

Devuelve únicamente el ID y nombre de las zonas. Ideal para llenar selects o dropdowns en el Frontend de forma ligera (ej. asociar brigada a zona).

Example responses

200 Response

```
[
  {
    "id_zona": "f93aee30-e9ed-4b8b-a1ad-e843e48e671e",
    "nombre": "string"
  }
]
```

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	Inline

Response Schema

Status Code **200**

Response Listar Zonas Simple Api V1 Zonas Simple Get

Name	Type	Required	Restrictions	Description
Response Listar Zonas Simple Api V1 Zonas Simple Get	[ZonaSimple]	false	none	none
» ZonaSimple	ZonaSimple	false	none	none
»» id_zona	string(uuid)	true	none	none
»» nombre	string	true	none	none

This operation does not require authentication

listar_riesgo_publico_api_v1_zonas_riesgo_publico_get

Code samples

```
# You can also use wget
curl -X GET /ml/api/v1/zonas/riesgo-publico \
-H 'Accept: application/json'
```

```
GET /ml/api/v1/zonas/riesgo-publico HTTP/1.1
```

```
Accept: application/json
```

```
const headers = {
  'Accept': 'application/json'
};

fetch('/ml/api/v1/zonas/riesgo-publico',
{
  method: 'GET',

  headers: headers
})
.then(function(res) {
  return res.json();
}).then(function(body) {
  console.log(body);
});
```

```
require 'rest-client'
require 'json'

headers = {
  'Accept' => 'application/json'
}

result = RestClient.get '/ml/api/v1/zonas/riesgo-publico',
  params: {
  }, headers: headers

p JSON.parse(result)
```

```
import requests
headers = {
  'Accept': 'application/json'
}
```

```
r = requests.get('/ml/api/v1/zonas/riesgo-publico', headers = headers)

print(r.json())
```

```
<?php

require 'vendor/autoload.php';

$headers = array(
    'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
    $response = $client->request('GET', '/ml/api/v1/zonas/riesgo-publico', array(
        'headers' => $headers,
        'json' => $request_body,
    ));
    print_r($response->getBody()->getContents());
}
catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...
```

```
URL obj = new URL("/ml/api/v1/zonas/riesgo-publico");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("GET");
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();
System.out.println(response.toString());
```

```
package main

import (
    "bytes"
    "net/http"
```

```

)

func main() {

    headers := map[string][]string{
        "Accept": []string{"application/json"},
    }

    data := bytes.NewBuffer([]byte{jsonReq})
    req, err := http.NewRequest("GET", "/ml/api/v1/zonas/riesgo-publico", data)
    req.Header = headers

    client := &http.Client{}
    resp, err := client.Do(req)
    // ...

}

```

GET /api/v1/zonas/riesgo-publico

Listar Riesgo Publico

Devuelve la lista de zonas con su último nivel de riesgo, ideal para el portal ciudadano sin exponer datos técnicos (HU23).

Example responses

200 Response

```

[
  {
    "nombre": "string",
    "nivel_riesgo": "string"
  }
]

```

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	Inline

Response Schema

Status Code **200**

Response Listar Riesgo Publico Api V1 Zonas Riesgo Publico Get

Name	Type	Required	Restrictions	Description
Response Listar Riesgo Publico Api V1 Zonas Riesgo Publico Get	[ZonaRiesgoPublico]	false	none	none
» ZonaRiesgoPublico	ZonaRiesgoPublico	false	none	none
»» nombre	string	true	none	none

»» nivel_riesgo	string	true	none	none
-----------------	--------	------	------	------

This operation does not require authentication

listar_zonas_api_v1_zonas__get

Code samples

```
# You can also use wget
curl -X GET /ml/api/v1/zonas/ \
  -H 'Accept: application/json'
```

```
GET /ml/api/v1/zonas/ HTTP/1.1
```

```
Accept: application/json
```

```
const headers = {
  'Accept': 'application/json'
};

fetch('/ml/api/v1/zonas/',
{
  method: 'GET',

  headers: headers
})
.then(function(res) {
  return res.json();
}).then(function(body) {
  console.log(body);
});
```

```
require 'rest-client'
require 'json'

headers = {
  'Accept' => 'application/json'
}

result = RestClient.get '/ml/api/v1/zonas/',
  params: {
  }, headers: headers

p JSON.parse(result)
```

```
import requests
headers = {
  'Accept': 'application/json'
```

```

}

r = requests.get('/ml/api/v1/zonas/', headers = headers)

print(r.json())

```

```

<?php

require 'vendor/autoload.php';

$headers = array(
    'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
    $response = $client->request('GET', '/ml/api/v1/zonas/', array(
        'headers' => $headers,
        'json' => $request_body,
    ));
    print_r($response->getBody()->getContents());
}
catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...

```

```

URL obj = new URL("/ml/api/v1/zonas/");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("GET");
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();
System.out.println(response.toString());

```

```

package main

import (

```

```
        "bytes"
        "net/http"
    )

    func main() {

        headers := map[string][]string{
            "Accept": []string{"application/json"},
        }

        data := bytes.NewBuffer([]byte{jsonReq})
        req, err := http.NewRequest("GET", "/ml/api/v1/zonas/", data)
        req.Header = headers

        client := &http.Client{}
        resp, err := client.Do(req)
        // ...
    }
```

GET /api/v1/zonas/

Listar Zonas

Devuelve la lista de todas las Zonas Protegidas registradas en PostGIS, incluyendo su geometría GeoJSON.

Example responses

200 Response

```
[
  {
    "id_zona": "f93aee30-e9ed-4b8b-a1ad-e843e48e671e",
    "nombre": "string",
    "area_hectareas": 0,
    "geojson": "string"
  }
]
```

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	Inline

Response Schema

Status Code **200**

Response Listar Zonas Api V1 Zonas Get

Name	Type	Required	Restrictions	Description
Response Listar Zonas Api V1 Zonas Get	[ZonaResponse]	false	none	none
» ZonaResponse	ZonaResponse	false	none	none

»» id_zona	string(uuid)	true	none	none
»» nombre	string	true	none	none
»» area_hectareas	number	true	none	none
»» geojson	any	false	none	none

anyOf

Name	Type	Required	Restrictions	Description
»»» <i>anonymous</i>	string	false	none	none

or

Name	Type	Required	Restrictions	Description
»»» <i>anonymous</i>	null	false	none	none

This operation does not require authentication

crear_zona_protegida_api_v1_zonas_post

Code samples

```
# You can also use wget
curl -X POST /ml/api/v1/zonas/ \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json'
```

```
POST /ml/api/v1/zonas/ HTTP/1.1
```

```
Content-Type: application/json
Accept: application/json
```

```
const inputBody = '{
  "nombre": "string",
  "geojson_polygon": {}
}';
const headers = {
  'Content-Type': 'application/json',
  'Accept': 'application/json'
};

fetch('/ml/api/v1/zonas/',
{
  method: 'POST',
  body: inputBody,
  headers: headers
})
.then(function(res) {
```



```
    return res.json();
  }).then(function(body) {
    console.log(body);
  });
```

```
require 'rest-client'
require 'json'

headers = {
  'Content-Type' => 'application/json',
  'Accept' => 'application/json'
}

result = RestClient.post '/ml/api/v1/zonas/',
  params: {
    }, headers: headers

p JSON.parse(result)
```

```
import requests
headers = {
  'Content-Type': 'application/json',
  'Accept': 'application/json'
}

r = requests.post('/ml/api/v1/zonas/', headers = headers)

print(r.json())
```

```
<?php

require 'vendor/autoload.php';

$headers = array(
  'Content-Type' => 'application/json',
  'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
    $response = $client->request('POST', '/ml/api/v1/zonas/', array(
        'headers' => $headers,
        'json' => $request_body,
    ));
    print_r($response->getBody()->getContents());
}
```

```

catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...

```

```

URL obj = new URL("/ml/api/v1/zonas/");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("POST");
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();
System.out.println(response.toString());

```

```

package main

import (
    "bytes"
    "net/http"
)

func main() {

    headers := map[string][]string{
        "Content-Type": []string{"application/json"},
        "Accept": []string{"application/json"},
    }

    data := bytes.NewBuffer([]byte{jsonReq})
    req, err := http.NewRequest("POST", "/ml/api/v1/zonas/", data)
    req.Header = headers

    client := &http.Client{}
    resp, err := client.Do(req)
    // ...
}

```

POST /api/v1/zonas/

Crear Zona Protegida

Crea una nueva zona de monitoreo recibiendo el GeoJSON dibujado por el Administrador (HU22).

Body parameter

```
{
  "nombre": "string",
  "geojson_polygon": {}
}
```

Parameters

Name	In	Type	Required	Description
body	body	ZonaCreate	true	none

Example responses

200 Response

```
{
  "id_zona": "f93aee30-e9ed-4b8b-a1ad-e843e48e671e",
  "nombre": "string",
  "area_hectareas": 0,
  "geojson": "string"
}
```

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	ZonaResponse
422	Unprocessable Entity	Validation Error	HTTPValidationError

This operation does not require authentication

obtener_clima_zona_api_v1_zonas__id_zona__clima_get

Code samples

```
# You can also use wget
curl -X GET /ml/api/v1/zonas/{id_zona}/clima \
  -H 'Accept: application/json'
```

```
GET /ml/api/v1/zonas/{id_zona}/clima HTTP/1.1
```

```
Accept: application/json
```

```
const headers = {
  'Accept': 'application/json'
};

fetch('/ml/api/v1/zonas/{id_zona}/clima',
```

```

{
  method: 'GET',

  headers: headers
})
.then(function(res) {
  return res.json();
}).then(function(body) {
  console.log(body);
});

```

```

require 'rest-client'
require 'json'

headers = {
  'Accept' => 'application/json'
}

result = RestClient.get '/ml/api/v1/zonas/{id_zona}/clima',
  params: {
  }, headers: headers

p JSON.parse(result)

```

```

import requests
headers = {
  'Accept': 'application/json'
}

r = requests.get('/ml/api/v1/zonas/{id_zona}/clima', headers = headers)

print(r.json())

```

```

<?php

require 'vendor/autoload.php';

$headers = array(
  'Accept' => 'application/json',
);

$client = new \GuzzleHttp\Client();

// Define array of request body.
$request_body = array();

try {
  $response = $client->request('GET', '/ml/api/v1/zonas/{id_zona}/clima', array(
    'headers' => $headers,
    'json' => $request_body,
  ))
}

```

```

    );
    print_r($response->getBody()->getContents());
}
catch (\GuzzleHttp\Exception\BadResponseException $e) {
    // handle exception or api errors.
    print_r($e->getMessage());
}

// ...

```

```

URL obj = new URL("/ml/api/v1/zonas/{id_zona}/clima");
URLConnection con = (URLConnection) obj.openConnection();
con.setRequestMethod("GET");
int responseCode = con.getResponseCode();
BufferedReader in = new BufferedReader(
    new InputStreamReader(con.getInputStream()));
String inputLine;
StringBuffer response = new StringBuffer();
while ((inputLine = in.readLine()) != null) {
    response.append(inputLine);
}
in.close();
System.out.println(response.toString());

```

```

package main

import (
    "bytes"
    "net/http"
)

func main() {

    headers := map[string][]string{
        "Accept": []string{"application/json"},
    }

    data := bytes.NewBuffer([]byte{jsonReq})
    req, err := http.NewRequest("GET", "/ml/api/v1/zonas/{id_zona}/clima", data)
    req.Header = headers

    client := &http.Client{}
    resp, err := client.Do(req)
    // ...
}

```

```
GET /api/v1/zonas/{id_zona}/clima
```

Obtener Clima Zona

Devuelve las últimas condiciones meteorológicas registradas para una zona (HU28). Se expone para que el backend operativo lo consuma en su portal ciudadano.

Parameters

Name	In	Type	Required	Description
id_zona	path	string	true	none

Example responses

200 Response

null

Responses

Status	Meaning	Description	Schema
200	OK	Successful Response	Inline
422	Unprocessable Entity	Validation Error	HTTPValidationError

Response Schema

This operation does not require authentication

Schemas

CondicionUmbra

```
{
  "temp": 0,
  "hum": 0
}
```

CondicionUmbra

Properties

Name	Type	Required	Restrictions	Description
temp	number	true	none	none
hum	number	true	none	none

HTTPValidationError

```
{
  "detail": [
    {
      "loc": [
        "string"
      ],
      "msg": "string",
    }
  ]
}
```

```
    "type": "string",
    "input": null,
    "ctx": {}
  }
]
```

HTTPValidationError

Properties

Name	Type	Required	Restrictions	Description
detail	[ValidationError]	false	none	none

PredictionRequest

```
{
  "temperatura": 35.5,
  "humedad": 30,
  "viento": 15,
  "precipitacion": 0,
  "id_zona": "f93aee30-e9ed-4b8b-a1ad-e843e48e671e"
}
```

PredictionRequest

Properties

Name	Type	Required	Restrictions	Description
temperatura	number	true	none	Temperatura máxima en °C
humedad	number	true	none	Humedad relativa media en %
viento	number	true	none	Velocidad del viento máxima en km/h
precipitacion	number	true	none	Precipitación acumulada en mm
id_zona	any	false	none	UUID de la zona protegida (opcional)

anyOf

Name	Type	Required	Restrictions	Description
» <i>anonymous</i>	string(uuid)	false	none	none

or

Name	Type	Required	Restrictions	Description
» <i>anonymous</i>	null	false	none	none

PredictionResponse

```
{
  "id_prediccion": "c641af1d-4c73-4a4a-863c-f8082ef3d52a",
  "id_zona": "f93aee30-e9ed-4b8b-a1ad-e843e48e671e",
  "nivel_riesgo": "string",
  "detalles_modelos": {},
  "directiva_accion": "string",
  "fecha_evaluacion": "2019-08-24T14:15:22Z"
}
```

PredictionResponse

Properties

Name	Type	Required	Restrictions	Description
id_prediccion	string(uuid)	true	none	none
id_zona	any	false	none	none

anyOf

Name	Type	Required	Restrictions	Description
» <i>anonymous</i>	string(uuid)	false	none	none

or

Name	Type	Required	Restrictions	Description
» <i>anonymous</i>	null	false	none	none

continued

Name	Type	Required	Restrictions	Description
nivel_riesgo	string	true	none	Bajo, Medio, Alto, o Crítico
detalles_modelos	object	true	none	Resultados crudos de KMeans e Isolation Forest
directiva_accion	string	true	none	Texto generado por LangChain con la recomendación de mitigación
fecha_evaluacion	string(date-time)	true	none	none

UmbralesConfig

```
{
  "critico": {
    "temp": 0,
```



```
    "hum": 0
  },
  "medio": {
    "temp": 0,
    "hum": 0
  }
}
```

UmbralesConfig

Properties

Name	Type	Required	Restrictions	Description
critico	CondicionUmbral	true	none	none
medio	CondicionUmbral	true	none	none

ValidationError

```
{
  "loc": [
    "string"
  ],
  "msg": "string",
  "type": "string",
  "input": null,
  "ctx": {}
}
```

ValidationError

Properties

Name	Type	Required	Restrictions	Description
loc	[anyOf]	true	none	none

anyOf

Name	Type	Required	Restrictions	Description
» <i>anonymous</i>	string	false	none	none

or

Name	Type	Required	Restrictions	Description
» <i>anonymous</i>	integer	false	none	none

continued

Name	Type	Required	Restrictions	Description
------	------	----------	--------------	-------------

msg	string	true	none	none
type	string	true	none	none
input	any	false	none	none
ctx	object	false	none	none

ZonaCreate

```
{
  "nombre": "string",
  "geojson_polygon": {}
}
```

ZonaCreate

Properties

Name	Type	Required	Restrictions	Description
nombre	string	true	none	none
geojson_polygon	object	true	none	Diccionario GeoJSON tipo Polygon o MultiPolygon

ZonaResponse

```
{
  "id_zona": "f93aee30-e9ed-4b8b-a1ad-e843e48e671e",
  "nombre": "string",
  "area_hectareas": 0,
  "geojson": "string"
}
```

ZonaResponse

Properties

Name	Type	Required	Restrictions	Description
id_zona	string(uuid)	true	none	none
nombre	string	true	none	none
area_hectareas	number	true	none	none
geojson	any	false	none	none

anyOf

Name	Type	Required	Restrictions	Description
------	------	----------	--------------	-------------

» <i>anonymous</i>	string	false	none	none
--------------------	--------	-------	------	------

or

Name	Type	Required	Restrictions	Description
» <i>anonymous</i>	null	false	none	none

ZonaRiesgoPublico

```
{
  "nombre": "string",
  "nivel_riesgo": "string"
}
```

ZonaRiesgoPublico

Properties

Name	Type	Required	Restrictions	Description
nombre	string	true	none	none
nivel_riesgo	string	true	none	none

ZonaSimple

```
{
  "id_zona": "f93aee30-e9ed-4b8b-a1ad-e843e48e671e",
  "nombre": "string"
}
```

ZonaSimple

Properties

Name	Type	Required	Restrictions	Description
id_zona	string(uuid)	true	none	none
nombre	string	true	none	none